Caderno de Atividades — JavaScript

Desenvolver a capacidade de interpretar problemas, construir algoritmos, utilizar estruturas da linguagem JavaScript e testar soluções de forma independente.

Orientações gerais

1. Leia o enunciado antes de começar a programar.
2. Utilize o VS Code e execute os códigos com Node.js.
3. Quando necessário, utilize readline-sync para receber dados digitados pelo usuário.
4. Não se limite a fazer o código funcionar: teste diferentes entradas e verifique os resultados.
5. Nos exercícios de depuração, identifique o erro, explique por que ele ocorre e apresente o código corrigido.
6. Nas questões de completar código, recrie no Visual studio code e preencha somente as partes solicitadas, preservando a estrutura fornecida.
7. Nas atividades de criação, desenvolva sua própria solução sem copiar um código pronto.

Parte 1 — Tipos de dados, variáveis e operadores (Nível 1 • Fundamentos da programação)

Atividade 1 — Identificando os tipos de dados

Observe as informações a seguir e identifique o tipo de dado mais adequado para armazenar cada uma delas: string, number ou boolean.

a) Nome de um personagem de jogo.

b) Quantidade de vidas restantes em um jogo.

c) Informação que indica se uma porta está aberta.

d) Preço de um produto.

e) Nome de uma cidade.

f) Indicação de que um computador está ligado.

g) Distância percorrida em quilômetros.

h) Código de identificação que pode conter zeros à esquerda, como "00421".

Atividade 2 — Complete o código

Complete as lacunas para que o programa apresente corretamente as informações de um dispositivo eletrônico.

```
const dispositivo = "Notebook";

let memoria = 16;
```

```
let armazenamento = 512;

let ligado = true;


console.log(`Dispositivo: ${_________}`);

console.log(`Memória RAM: ${_________} GB`);

console.log(`Armazenamento: ${_________} GB`);

console.log(`Está ligado? ${_________}`);
```

Tarefa extra: acrescente uma variável que armazene o preço do dispositivo e apresente essa informação utilizando uma template string.

Atividade 3 — Prevendo o resultado

Sem executar o programa, escreva o resultado de cada operação.

```
console.log(18 + 7);

console.log(30 - 12);

console.log(6 * 8);

console.log(45 / 5);

console.log(17 % 4);

console.log(3 ** 3);

console.log("Java" + "Script");

console.log("10" + 5);

console.log("10" - 5);
```

Depois, execute o código e compare suas respostas. Explique por que a concatenação de "10" com 5 produz um resultado diferente da subtração entre esses valores.

Atividade 4 — Encontrando os erros

O programa abaixo deveria calcular o custo total de uma viagem, mas apresenta problemas.

```
const distancia = "120";

const consumo = 12;

const precoCombustivel = 6.20;


let litros = distancia / consumo;

let custo = litros + precoCombustivel;


console.log("Litros necessários: " + litros);
```

console.log("Custo da viagem: R$ " + custo);

Tarefas:

- Identifique o problema relacionado ao tipo de dado de distancia.
- Verifique se a operação utilizada para calcular o custo está correta.
- Corrija o programa.
- Apresente o consumo estimado em litros e o custo total da viagem.

Considere que o veículo percorre 12 km por litro e que o preço do combustível é R$ 6,20 por litro.

Atividade 5 — Criando um conversor de tempo

Desenvolva um programa que receba uma quantidade inteira de segundos e converta esse valor em horas, minutos e segundos restantes.

Exemplo: para uma entrada de 3672 segundos, o programa deverá apresentar 1 hora, 1 minuto e 12 segundos.

Requisitos:

- Utilizar variáveis.
- Utilizar operadores aritméticos.
- Utilizar divisão e resto da divisão.
- Apresentar o resultado em uma mensagem organizada.

Atividade 6 — Operadores relacionais e lógicos

Complete a tabela indicando se cada expressão resulta em true ou false.

Expressão	Resultado
15 > 10	_____
8 === "8"	_____
12 <= 12	_____
!(5 > 9)	_____
(10 > 5) && (3 < 8)	_____
(4 > 10) || (7 === 7)	_____
(6 !== 6) || (2 <= 2)	_____
(9 > 3) && !(4 === "4")	_____

Em seguida, explique com suas palavras a diferença entre os operadores && e ||.

Parte 2 — Estruturas condicionais (Nível 1 e 2 • Tomada de decisão)

Atividade 7 — Complete a classificação

Uma plataforma de jogos precisa classificar a idade de seus usuários para uma atividade fictícia.

Complete o código para apresentar uma das seguintes classificações:

- Menor de 13 anos: "Categoria infantil".
- De 13 a 17 anos: "Categoria juvenil".
- 18 anos ou mais: "Categoria adulta".
- Idade negativa: "Idade inválida".

```
const idade = 16;


if (_____) {
   console.log("Idade inválida");
} else if (_____) {
   console.log("Categoria infantil");
} else if (_____) {
   console.log("Categoria juvenil");
} else {
   console.log("Categoria adulta");
}
```

Teste com as idades -1, 10, 16 e 25.

Atividade 8 — Corrigindo uma condição

O código deveria permitir a entrada de uma pessoa em uma área de recreação somente quando ela tiver pelo menos 12 anos e estiver acompanhada por um responsável.

```
const idade = 10;
const acompanhado = false;


if (idade >= 12 || acompanhado === true) {
   console.log("Entrada permitida");
} else {
   console.log("Entrada não permitida");
}
```

Tarefas:

a) Identifique o erro lógico.

b) Explique por que uma pessoa com 10 anos e sem acompanhante poderia receber uma mensagem incorreta se a condição fosse interpretada de maneira inadequada.

c) Corrija a condição para que as duas exigências sejam atendidas simultaneamente.

d) Teste as quatro combinações possíveis de idade suficiente ou insuficiente e presença ou ausência de responsável.

Atividade 9 — Criando um sistema de acesso

Uma sala de informática possui regras de acesso. Desenvolva um programa que receba o nome do usuário e a quantidade de horas de treinamento concluídas.

O acesso será liberado quando:

- O nome não estiver vazio.
- O usuário tiver concluído pelo menos 4 horas de treinamento.

O programa deverá informar se o acesso foi liberado ou negado e explicar o motivo quando houver uma restrição.

Utilize if, else if e else, conforme considerar adequado.

Atividade 10 — Menu com switch case

Uma aplicação de mídia oferece as seguintes opções:

Código	**Ação**
1	Reproduzir música
2	Pausar reprodução
3	Avançar faixa
4	Encerrar aplicação

Complete o programa.

```
const readline = require("readline-sync");

let opcao = Number(
   readline.question("Escolha uma opcao: ")
);


switch (_____) {
   case 1:
```

```
        console.log("_____");
        break;

    case 2:
        console.log("_____");
        break;

    case 3:
        console.log("_____");
        break;

    case 4:
        console.log("_____");
        break;

    default:
        console.log("_____");
}
```

Preencha as lacunas, execute o código e teste todas as opções, inclusive um código inexistente.

Atividade 11 — Criando um calculador de frete

Uma loja virtual utiliza a seguinte regra para calcular o frete:

- Compras com valor inferior a R$ 100,00: frete de R$ 20,00.
- Compras de R$ 100,00 até R$ 249,99: frete de R$ 12,00.
- Compras a partir de R$ 250,00: frete grátis.
- Valor de compra negativo: entrada inválida.

Crie um programa que receba o valor da compra, calcule o frete e apresente o valor final, incluindo a compra e o frete.

Teste valores próximos dos limites: 99.99, 100, 249.99 e 250.

Atividade 12 — Diagnóstico de um computador

Uma ferramenta de suporte precisa apresentar uma recomendação a partir de três informações:

- O computador está ligado?
- A conexão de rede está funcionando?
- O usuário consegue acessar o sistema?

Crie uma sequência de condições que produza recomendações diferentes, como verificar a alimentação elétrica, verificar a rede, reiniciar o aplicativo ou encaminhar o problema para suporte técnico.

Você deverá utilizar operadores lógicos e, no mínimo, quatro caminhos possíveis de decisão.

Não existe uma única solução: o importante é que as condições sejam coerentes e que todos os cenários previstos sejam tratados.

Parte 3 — Estruturas de repetição (Nível 2 • Repetição, contagem e acumulação)

Atividade 13 — Complete a contagem

O programa deve apresentar os números de 1 até 20.

```
let contador = 1;


while (_____) {
   console.log(contador);
   _____;
}
```

Complete as lacunas e explique o que aconteceria se o contador não fosse incrementado.

Atividade 14 — Encontrando um loop infinito

Analise o código:

```
let numero = 10;


while (numero >= 1) {
   console.log(numero);
   numero++;
}
```

a) Qual era provavelmente a intenção do programador?

b) Por que o programa não termina normalmente?

c) Corrija o código para realizar uma contagem regressiva de 10 até 1.

d) Modifique o programa para, ao final, apresentar a mensagem "Contagem finalizada!".

Atividade 15 — Somando números pares

Crie um programa que utilize for para percorrer os números de 1 até 100 e calcular a soma de todos os números pares desse intervalo.

O programa deverá apresentar:

- A soma final.
- A quantidade de números pares encontrados.

Não informe os resultados manualmente: ambos devem ser calculados pelo código.

Atividade 16 — Repetição com validação

Complete o programa para solicitar uma senha até que o usuário digite a senha correta.

```
const readline = require("readline-sync");

const senhaCorreta = "js2026";


let senhaDigitada = "";


while (_____) {
  senhaDigitada = readline.question("Digite a senha: ");


  if (_____) {
    console.log("Senha incorreta. Tente novamente.");
  }
}


console.log("Acesso autorizado!");
```

Teste com uma senha errada, duas senhas erradas e, por fim, a senha correta.

Atenção: este é um exercício didático de controle de fluxo, não uma implementação de autenticação segura.

Atividade 17 — Diferenças entre for, while e do...while

Considere a necessidade de apresentar os números de 1 a 5.

a) Resolva o problema utilizando for.

b) Reescreva utilizando while.

c) Reescreva utilizando do...while.

d) Explique em qual situação cada estrutura pode ser mais conveniente.

e) No do...while, o que acontece quando a condição é falsa já na primeira verificação?

Atividade 18 — Registro de distâncias

Crie um programa que permita registrar as distâncias percorridas em diferentes trechos de uma viagem.

O usuário deverá informar as distâncias em quilômetros, uma por vez. A digitação do valor 0 encerrará o registro e não deverá ser somada.

Ao final, apresente:

- Quantos trechos foram registrados.
- A distância total percorrida.
- A maior distância informada.

Requisitos: utilize while ou while (true), uma variável acumuladora e uma variável contadora. Trate valores negativos como inválidos e não os inclua nos cálculos.

Atividade 19 — Desenhando padrões com repetição

Crie um programa que apresente o seguinte padrão utilizando repetições:

*

**

***

****

*****

Depois, modifique o código para apresentar o padrão invertido:

*****

****

***

**

*

Por fim, crie um terceiro padrão em que cada linha contenha o número da própria linha repetido essa quantidade de vezes.

Exemplo:

1

22

333

4444

55555

Utilize repetições aninhadas, isto é, uma estrutura de repetição dentro de outra.

Parte 4 — Arrays (Nível 2 • Coleções, índices e percursos)

Atividade 20 — Interpretando posições de um array

Considere o código:

```
let ferramentas = [
   "VS Code",
   "GitHub",
   "MySQL Workbench",
   "Node.js",
   "Postman"
];

console.log(ferramentas[0]);
console.log(ferramentas[2]);
console.log(ferramentas[4]);
console.log(ferramentas.length);
```

Responda antes de executar:

a) O que será apresentado em cada linha?

b) Quantos elementos existem no array?

c) Qual é o índice do último elemento?

d) O que acontece ao executar console.log(ferramentas[5])?

e) Como substituir "Postman" por "Insomnia"?

Atividade 21 — Complete as operações

Complete o código para realizar as operações solicitadas.

```
let linguagens = ["JavaScript", "Python", "Java"];

// 1. Adicione "C#" ao final.
_____;
```

// 2. Adicione "TypeScript" ao início.

_____;

// 3. Remova o último elemento.

_____;

// 4. Apresente a quantidade de elementos.

_____;

// 5. Apresente o conteúdo completo do array.

_____;

Depois, escreva em uma folha o conteúdo do array após cada operação.

Atividade 22 — Corrigindo o cadastro de participantes

O código abaixo deveria apresentar todos os participantes de um evento, mas não funciona corretamente.

```
let participantes = [
   "Ana",
   "Bruno",
   "Carla",
   "Daniel",
   "Eduarda"
];

for (let i = 1; i <= participantes.length; i++) {
   console.log(participantes[i]);
}
```

a) Identifique os dois problemas na estrutura de repetição.

b) Corrija o código.

c) Modifique a solução para apresentar cada participante com seu número de ordem, começando por 1.

Exemplo: 1 - Ana.

Atividade 23 — Pesquisa em um array

Uma biblioteca de jogos mantém a seguinte lista:

```
let jogos = [
  "Minecraft",
  "Terraria",
  "Stardew Valley",
  "Hollow Knight",
  "Celeste"
];
```

Desenvolva um programa que solicite o nome de um jogo e percorra o array para verificar se ele está cadastrado.

O resultado deverá indicar se o jogo foi encontrado. Se for encontrado, apresente também sua posição na lista.

Regras:

- A pesquisa deve ser realizada com uma estrutura de repetição.
- Utilize uma variável de controle para identificar se houve correspondência.
- Não utilize objetos.
- Considere que os nomes digitados precisam corresponder exatamente aos nomes cadastrados.

Atividade 24 — Contando ocorrências

Uma lista registra os tipos de equipamentos solicitados para manutenção:

```
let equipamentos = [
  "Mouse",
  "Teclado",
  "Monitor",
  "Mouse",
  "Headset",
  "Mouse",
  "Teclado"
];
```

Crie um programa que conte quantas vezes um equipamento aparece no array.

O usuário deverá digitar o nome do equipamento pesquisado. O programa deverá apresentar a quantidade de ocorrências encontradas.

Desafio adicional: modifique a solução para apresentar a quantidade de ocorrências de "Mouse", "Teclado" e "Monitor" em uma única execução.

Atividade 25 — Analisando temperaturas

Uma estação meteorológica registrou as seguintes temperaturas:

let temperaturas = [28, 31, 29, 34, 27, 33, 30];

Desenvolva um programa que percorra o array e calcule:

- A maior temperatura.
- A menor temperatura.
- A soma de todas as temperaturas.
- A média aritmética.
- A quantidade de temperaturas acima de 30 °C.

Utilize variáveis auxiliares e estruturas de repetição. Não utilize funções prontas como Math.max() ou Math.min() para encontrar os extremos.

Atividade 26 — Removendo elementos por posição

Considere o array:

```
let tarefas = [
  "Estudar JavaScript",
  "Praticar arrays",
  "Revisar funções",
  "Estudar objetos",
  "Resolver exercícios"
];
```

Crie um programa que:

1. Apresente as tarefas cadastradas.
2. Solicite o índice da tarefa que deverá ser removida.
3. Verifique se o índice é válido.
4. Remova a tarefa selecionada.
5. Apresente o array atualizado.

Utilize splice() para remover o elemento. Não permita a remoção quando o índice informado estiver fora do intervalo válido.

Atividade 27 — Criando um sistema de favoritos

Desenvolva um pequeno sistema para gerenciar uma lista de filmes favoritos.

O programa deverá apresentar um menu com as seguintes opções:

1. Adicionar filme.
2. Listar filmes.
3. Remover filme por posição.
4. Pesquisar filme.
5. Encerrar o programa.

Requisitos técnicos:

- Utilize um array para armazenar os títulos.
- Utilize switch case para tratar as opções.
- Utilize uma estrutura de repetição para manter o menu ativo.
- Verifique se a lista está vazia antes de listar ou remover elementos.
- Não permita adicionar títulos vazios.
- Informe quando uma pesquisa não encontrar o filme.

A atividade deve ser implementada sem objetos e sem funções, para concentrar a prática nos arrays, condicionais e repetições.

Parte 5 — Funções (Nível 2 e 3 • Organização e reutilização de código)

Atividade 28 — Parâmetros e retorno

Complete as funções a seguir.

```
function calcularAreaRetangulo(largura, altura) {
   let area = _____;
   return _____;
}


function calcularPerimetroRetangulo(largura, altura) {
   let perimetro = _____;
   return _____;
}
```

```
let areaCalculada = calcularAreaRetangulo(8, 5);

let perimetroCalculado = calcularPerimetroRetangulo(8, 5);


console.log(`Área: ${areaCalculada}`);

console.log(`Perímetro: ${perimetroCalculado}`);
```

Após completar, explique a diferença entre os parâmetros largura e altura e os argumentos 8 e 5 utilizados na chamada.

Atividade 29 — Identificando o problema com return

Observe o programa:

```
function calcularDobro(numero) {
   console.log(numero * 2);
}


let resultado = calcularDobro(10);


console.log(`Resultado armazenado: ${resultado}`);
```

a) O que será apresentado no terminal?

b) Por que a variável resultado não armazena o dobro de 10?

c) Reescreva a função utilizando return.

d) Faça o programa apresentar o dobro e o triplo do mesmo número, utilizando a função para obter os resultados.

Atividade 30 — Criando funções de conversão

Desenvolva três funções:

- celsiusParaFahrenheit(celsius): converte uma temperatura Celsius para Fahrenheit.
- quilometrosParaMetros(quilometros): converte quilômetros para metros.
- minutosParaSegundos(minutos): converte minutos para segundos.

Cada função deverá receber um parâmetro e retornar o resultado calculado.

Depois, chame cada função com pelo menos três valores diferentes e apresente os resultados com template strings.

Pesquise as fórmulas de conversão e aplique ao seu código.

Atividade 31 — Encontrando o erro de escopo

Analise o código:

```
function calcularDistancia(velocidade, tempo) {
   let distancia = velocidade * tempo;
}

calcularDistancia(60, 2);

console.log(distancia);
```

a) Identifique o problema relacionado ao escopo da variável.

b) Explique por que a variável declarada dentro da função não está disponível diretamente fora dela.

c) Corrija a função para retornar a distância.

d) Armazene o resultado em uma variável e apresente a distância percorrida em quilômetros.

Atividade 32 — Função de validação

Uma aplicação de cadastro precisa verificar se um nome é válido.

Crie uma função chamada validarNome(nome) que retorne:

- true, se o nome contiver pelo menos 3 caracteres após a remoção dos espaços das extremidades.
- false, caso contrário.

Depois, utilize a função dentro de uma estrutura condicional para informar se o cadastro pode continuar.

Teste os valores "Ana", " João ", "Li" e "".

Dica: pesquise como funciona o método trim() de strings.

Atividade 33 — Funções com arrays

Crie as seguintes funções para trabalhar com uma lista de números:

- calcularSoma(numeros): retorna a soma dos elementos.
- contarPares(numeros): retorna a quantidade de números pares.
- encontrarMaior(numeros): retorna o maior elemento.
- buscarNumero(numeros, valor): retorna true se o valor existir no array e false caso contrário.

Utilize o seguinte array para testar:

let numeros = [12, 7, 24, 9, 31, 18, 5, 42];

As funções deverão percorrer o array utilizando estruturas de repetição. Não utilize métodos prontos que resolvam diretamente os cálculos solicitados.

Atividade 34 — Reescrevendo com arrow functions

Converta as funções tradicionais abaixo para arrow functions, mantendo o comportamento de cada uma.

```
function calcularCubo(numero) {
   return numero * numero * numero;
}


function verificarPar(numero) {
   return numero % 2 === 0;
}


function criarMensagem(nome) {
   return `Bem-vindo, ${nome}!`;
}
```

Depois:

a) Utilize cada função em uma chamada.

b) Reescreva as funções que possuem apenas uma expressão de retorno usando a forma reduzida.

c) Explique a diferença visual entre uma função tradicional e uma arrow function.

Atividade 35 — Construindo um menu com funções

Uma aplicação de organização pessoal deverá permitir que o usuário:

1. Consulte uma mensagem de boas-vindas.
2. Converta minutos em segundos.
3. Verifique se um número é par.
4. Encerre a aplicação.

Implemente o sistema utilizando readline-sync, switch case, repetição e funções.

Regras:

- Crie uma função para cada operação principal.

- O menu deverá permanecer ativo até que o usuário escolha encerrar.
- As funções responsáveis por cálculos deverão retornar os resultados.
- A função principal de cada operação deverá ser chamada a partir do menu.
- Trate opções inválidas.

O objetivo é separar as responsabilidades em funções, evitando escrever todo o programa dentro de um único bloco de código.

Parte 6 — Objetos em JavaScript (Nível 3 • Propriedades, valores e manipulação de objetos)

Nesta parte, os desafios começam com objetos simples, sem exigir conteúdos que ainda não foram estudados. O foco está em compreender como agrupar informações relacionadas, acessar propriedades e modificar valores.

Atividade 36 — Criando um objeto

Considere as informações de um livro:

- Título: O Hobbit.
- Autor: J. R. R. Tolkien.
- Ano de publicação: 1937.
- Disponível para empréstimo: verdadeiro.

Crie um objeto chamado livro que armazene essas informações.

Depois:

a) Apresente o título.

b) Apresente o nome do autor.

c) Altere o ano de publicação para 1938.

d) Altere a disponibilidade para false.

e) Apresente uma mensagem com o título, o autor e a situação do empréstimo.

Utilize a notação de ponto para acessar e modificar as propriedades.

Atividade 37 — Complete o objeto

Complete o código para acessar e modificar as propriedades de um objeto.

```
let computador = {
   marca: "Lenovo",
   memoriaRAM: 8,
   armazenamento: 256,
   ligado: false
};
```

```
// Apresente a marca.
console.log(_____);

// Altere a memória RAM para 16 GB.
_____;

// Ligue o computador.
_____;

// Apresente todas as informações.
console.log(
   `Marca: ${computador.marca}
RAM: ${computador.memoriaRAM} GB
Armazenamento: ${computador.armazenamento} GB
Ligado: ${computador.ligado}`
);
```

Atividade 38 — Encontrando erros em objetos

Analise o código:

```
let jogo = {
   nome: "Hollow Knight",
   genero: "Aventura",
   ano: 2017
};

console.log(jogo.titulo);

jogo.genero = "Ação e aventura";

jogo.anoLancamento = 2020;
```

console.log(jogo);

O código é executado, mas há um problema na primeira consulta e uma possível inconsistência nas informações.

a) Por que jogo.titulo apresenta undefined?

b) Como acessar corretamente o nome do jogo?

c) A atribuição a jogo.anoLancamento provoca um erro? Explique.

d) Corrija o ano de lançamento considerando as informações fornecidas no objeto original.

e) Apresente o objeto corrigido.

Atividade 39 — Objeto e função

Crie um objeto chamado personagem com as propriedades:

- nome
- vida
- nivel
- experiencia

Depois, crie uma função chamada receberDano(personagem, dano) que reduza a vida do personagem pelo valor recebido.

Regras:

- A vida não pode ficar abaixo de zero.
- Se a vida chegar a zero, apresente a mensagem "Personagem derrotado!".
- Caso contrário, apresente a vida restante.
- A função deverá modificar a propriedade vida do objeto recebido.

Teste diferentes quantidades de dano e verifique o comportamento do programa.

Atividade 40 — Cadastro de equipamentos

Crie um objeto chamado equipamento para representar um equipamento de informática.

O objeto deverá armazenar:

- Nome.
- Marca.
- Preço.
- Quantidade em estoque.
- Situação de funcionamento.

Em seguida, desenvolva um programa que permita modificar a quantidade em estoque e a situação de funcionamento.

Ao final, apresente todas as informações do equipamento.

Atenção: utilize apenas um objeto simples nesta atividade. Não é necessário criar arrays de objetos.

Parte 7 — Desafios integradores (Nível 3 • Resolução de problemas completos)

Aqui, o objetivo é combinar vários conteúdos já estudados. Os enunciados descrevem o problema, mas não fornecem o código pronto.

Atividade 41 — Sistema de reserva de salas

Uma instituição de ensino precisa de um programa para controlar a reserva de suas salas de estudo.

O sistema deverá apresentar um menu com as opções:

1. Consultar salas.
2. Reservar uma sala.
3. Cancelar uma reserva.
4. Consultar quantidade de salas disponíveis.
5. Encerrar o sistema.

Para começar, considere estas salas:

let salas = ["Sala A", "Sala B", "Sala C", "Sala D"];

let reservadas = [false, true, false, false];

Cada posição do array reservadas corresponde à sala na mesma posição do array salas. O valor true indica que a sala está reservada, e false indica que está disponível.

Requisitos:

- Listar as salas com seus respectivos estados.
- Permitir reservar apenas salas disponíveis.
- Permitir cancelar apenas reservas existentes.
- Validar a posição escolhida pelo usuário.
- Contar e apresentar as salas disponíveis.
- Manter o menu ativo até a escolha de encerramento.

Conteúdos praticados: arrays, índices, operadores, condicionais, repetição, entrada de dados e switch case.

Atividade 42 — Sistema de controle de biblioteca

Uma pequena biblioteca deseja organizar seu catálogo de livros e controlar os empréstimos.

O sistema deverá começar com os seguintes dados:

let livros = [

```
  "O Hobbit",
  "Dom Casmurro",
  "A Revolução dos Bichos",
  "O Pequeno Príncipe"
];

let emprestados = [false, true, false, false];
```

Desenvolva um menu com as seguintes funcionalidades:

1. Listar todos os livros.
2. Pesquisar um livro pelo título.
3. Realizar um empréstimo.
4. Registrar uma devolução.
5. Consultar quantos livros estão disponíveis.
6. Encerrar o sistema.

Regras do sistema:

- Um livro emprestado não pode ser emprestado novamente.
- Um livro disponível não pode ser devolvido sem que tenha sido emprestado.
- A pesquisa deverá informar quando o título não estiver cadastrado.
- A listagem deverá mostrar o título e a situação de cada livro.
- As opções inválidas deverão receber uma mensagem de erro.

O sistema deve funcionar continuamente até o usuário escolher encerrar.

Restrição: resolva a atividade usando arrays paralelos, condicionais e repetições, sem objetos e sem funções. Essa restrição ajuda a consolidar os conteúdos anteriores.

Atividade 43 — Organizador de tarefas

Desenvolva um sistema de terminal para registrar tarefas pessoais.

Cada tarefa terá um nome e uma situação: pendente ou concluída.

Utilize dois arrays paralelos:

```
let tarefas = [];

let concluidas = [];
```

Crie um menu com estas opções:

1. Cadastrar tarefa.

2. Listar tarefas.
3. Marcar tarefa como concluída.
4. Listar somente tarefas pendentes.
5. Excluir tarefa.
6. Apresentar um resumo das tarefas.
7. Encerrar o sistema.

Requisitos:

- Cada tarefa deve ser cadastrada com uma descrição não vazia.
- A situação inicial de uma nova tarefa deve ser pendente.
- A listagem deve apresentar o número da tarefa, a descrição e a situação.
- O usuário deve informar a posição da tarefa para concluí-la ou excluí-la.
- As posições informadas devem ser validadas.
- O resumo deve apresentar o total de tarefas, quantas estão concluídas e quantas estão pendentes.

A solução deverá utilizar arrays, operadores, condicionais, repetição e switch case.

Atividade 44 — Painel de monitoramento de ambiente

Uma equipe de suporte deseja monitorar as temperaturas registradas em diferentes salas de um laboratório.

Considere os dados iniciais:

let salas = ["Laboratório 1", "Laboratório 2", "Laboratório 3"];

let temperaturas = [23.5, 29.8, 26.2];

Desenvolva um programa que:

1. Apresente as temperaturas de todas as salas.
2. Permita registrar uma nova temperatura para uma sala.
3. Identifique a sala com a maior temperatura.
4. Calcule a temperatura média das salas.
5. Conte quantas salas estão com temperatura acima de 27 °C.
6. Informe se alguma sala ultrapassou 30 °C.
7. Encerre quando o usuário solicitar.

Regras:

- Os arrays devem manter as informações na mesma ordem.
- Os índices devem ser validados.

- A temperatura média deve ser calculada a partir dos valores atuais.
- A maior temperatura deve ser encontrada por meio de uma repetição.
- As alterações realizadas devem aparecer nas consultas seguintes.

Desafio de organização: depois de concluir a versão inicial, separe os cálculos em funções e compare as duas versões.

Atividade 45 — Catálogo de jogos

Uma comunidade de jogadores quer desenvolver um catálogo simples para organizar seus jogos favoritos.

Cada jogo possui nome, gênero e quantidade de horas jogadas.

Para esta atividade, utilize arrays separados:

let nomes = ["Minecraft", "Terraria", "Celeste"];

let generos = ["Sandbox", "Aventura", "Plataforma"];

let horas = [45, 20, 12];

Crie um sistema com estas opções:

1. Cadastrar jogo.
2. Listar jogos.
3. Pesquisar jogo por nome.
4. Atualizar horas jogadas.
5. Apresentar o jogo com mais horas.
6. Calcular o total de horas jogadas.
7. Encerrar.

Requisitos:

- Cada jogo novo deve ser incluído nos três arrays, mantendo a correspondência das posições.
- A pesquisa deve informar a posição do jogo encontrado.
- A atualização deve validar o índice e impedir a inclusão de horas negativas.
- A maior quantidade de horas e o total devem ser calculados pelo programa.
- A listagem deverá apresentar todas as informações de cada jogo.

Etapa de evolução: após concluir a primeira versão, reescreva o sistema utilizando um array de objetos, com cada jogo representado por um objeto contendo as propriedades nome, genero e horas.

Essa etapa permitirá comparar as duas formas de estruturar os mesmos dados.

Atividade 46 — Sistema de empréstimo de equipamentos

Uma instituição possui equipamentos que podem ser emprestados aos alunos durante as aulas práticas.

Cada equipamento possui nome, identificação e situação de disponibilidade.

Desenvolva uma aplicação que permita:

- Cadastrar equipamentos.
- Listar todos os equipamentos.
- Pesquisar um equipamento pela identificação.
- Registrar um empréstimo.
- Registrar uma devolução.
- Apresentar a quantidade de equipamentos disponíveis.
- Encerrar o programa.

Primeira versão: utilize arrays paralelos para armazenar os dados.

Segunda versão: utilize objetos simples para representar os equipamentos.

Requisitos das duas versões:

- Impedir empréstimos de equipamentos já emprestados.
- Impedir devoluções de equipamentos disponíveis.
- Validar a identificação informada.
- Apresentar mensagens claras de sucesso e erro.
- Manter os dados atualizados durante a execução.

Ao finalizar, escreva um pequeno parágrafo explicando qual versão foi mais fácil de compreender e manter.

Parte 8 — Laboratório de depuração (Nível 2 e 3 • Ler, investigar e corrigir códigos)

Nesta etapa, o aluno deverá atuar como um programador responsável por revisar códigos escritos por outra pessoa. Em cada atividade, não basta apresentar a correção: é necessário explicar o problema encontrado.

Atividade 47 — O contador que não conta corretamente

```
let contador = 0;


for (let i = 1; i < 10; i--) {
   contador++;
}
```

```
console.log(`Quantidade: ${contador}`);
```

Enunciado: o programa deveria contar de 1 até 10, mas não funciona como esperado.

1. Identifique o problema na repetição.
2. Corrija a condição e o incremento.
3. Faça o programa apresentar a quantidade de números percorridos.
4. Modifique o código para contar de 1 até 50.

Atividade 48 — A função que retorna o resultado errado

```
function calcularQuadrado(numero) {
  return numero + numero;
}

let resultado = calcularQuadrado(6);

console.log(`Quadrado: ${resultado}`);
```

Enunciado: a função deveria calcular o quadrado de um número, mas realiza outra operação.

Corrija a função, teste-a com três números diferentes e explique a diferença entre elevar um número ao quadrado e multiplicá-lo por dois.

Atividade 49 — O array com posições incorretas

```
let animais = ["Gato", "Cachorro", "Coelho", "Papagaio"];

for (let i = 0; i <= animais.length; i++) {
  console.log(`${i + 1} - ${animais[i]}`);
}
```

Enunciado: o programa deveria listar todos os animais, mas também apresenta um resultado indesejado.

Corrija o percurso do array e explique por que o último índice válido é diferente do tamanho do array.

Atividade 50 — A validação que falha

```
let idade = 17;

let possuiAutorizacao = false;
```

```
if (idade >= 18 && possuiAutorizacao === false) {
   console.log("Acesso liberado");
} else {
   console.log("Acesso negado");
}
```

Enunciado: uma aplicação de demonstração deve liberar o acesso somente para usuários com 18 anos ou mais. Entretanto, o programa precisa apresentar mensagens coerentes para cada situação.

Corrija a lógica para atender à regra estabelecida e explique por que a autorização não deve alterar o requisito mínimo de idade neste cenário.

Atividade 51 — A propriedade inexistente

```
let dispositivo = {
   nome: "Impressora",
   fabricante: "Epson",
   funcionando: true
};

if (dispositivo.funcionamento === true) {
   console.log("Equipamento funcionando");
} else {
   console.log("Verificar equipamento");
}
```

Enunciado: o programa deveria consultar a situação do equipamento, mas acessa uma propriedade incorreta.

Identifique o erro, corrija o código e explique como a linguagem se comporta quando tentamos acessar uma propriedade que não existe no objeto.

Atividade 52 — O sistema que perde informações

```
let nomes = ["Ana", "Bruno", "Carla"];
let status = [false, true, false];

nomes.push("Daniel");
status.push(true);
```

```
nomes.splice(1, 1);


console.log(nomes);

console.log(status);
```

Enunciado: o sistema representa pessoas e seus estados por meio de arrays paralelos. Após uma exclusão, os dados deixam de corresponder corretamente.

1. Execute o código e observe os arrays.
2. Identifique a causa do problema.
3. Corrija o processo de exclusão para remover a pessoa e o estado da mesma posição.
4. Explique por que os dois arrays precisam ser atualizados juntos.

Parte 9 — Desafios finais de criação (Nível 3 • Autonomia e planejamento de soluções)

Estas atividades devem ser desenvolvidas sem um código-base pronto. O aluno deverá interpretar a narrativa, planejar as etapas, escolher as estruturas necessárias e construir a solução.

Atividade 53 — Sistema de coleção musical

Uma pessoa deseja organizar sua coleção de músicas por meio de um programa de terminal.

Cada música possui título, artista e duração em segundos.

O sistema deverá permitir cadastrar músicas, listar a coleção, pesquisar títulos, atualizar a duração, remover músicas e apresentar a duração total da coleção.

Critérios técnicos:

- Utilizar arrays para armazenar os dados.
- Utilizar funções para separar as operações principais.
- Utilizar condicionais para validar as escolhas do usuário.
- Utilizar repetições para percorrer os registros e manter o menu ativo.
- Impedir títulos vazios e durações negativas.
- Apresentar a duração total em minutos e segundos.

Após concluir a versão inicial, reestruture o armazenamento utilizando objetos.